

Введение в нейронные сети

Руденко Ирина, разработчик-исследователь

План лекции

- 1 | Линейная классификация
- 2 | Однослойная нейронная сеть
- 3 | Многослойная нейронная сеть
- 4 | Метод обратного распространения ошибки
- 5 | Проблема переобучения: дропаут и пакетная нормализация

1



Линейная классификация.

Вспомним: Свойства линейных моделей

Достоинства:

- Легко интерпретируемы
- Легко вычислять
- Легко обучать (сегодня узнаем, как)

Недостатки:

- Подходят только для простых задач
- Плохо справляются с зависимыми признаками
- Сильно зависят от конкретной обучающей выборки

Вспомним: Линейная регрессия.

Матричная запись линейной регрессии

$\{X, y\}$ – датасет

$X = \{x_i\}_{i=1}^m$, $x_i \in \mathbb{R}^n$ - матрица признаков

$y = \{y_i\}_{i=1}^m$, $y_i \in \mathbb{R}$ - вектор ответов

$w = (w_0, w_1, w_2, \dots, w_n)$, $w_i \in \mathbb{R}$ - вектор весов

$$a(X) = Xw \approx y$$

$$a(X) = \begin{pmatrix} 1 & x_{1_1} & x_{1_2} & \dots & x_{1_n} \\ 1 & x_{2_1} & x_{2_2} & \dots & x_{2_n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{m_1} & x_{m_2} & \dots & x_{m_n} \end{pmatrix} \begin{pmatrix} w_0 \\ w_1 \\ w_2 \\ \vdots \\ w_n \end{pmatrix} \approx \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{pmatrix}$$

Вспомним: Линейная регрессия.

Функция потерь и функционал эмпирического риска.

$L(y_i, f(x_i))$ - функция потерь на паре (x_i, y_i)

MSE: $L(y_i, f(x_i)) = (y_i - f(x_i))^2$

MAE: $L(y_i, f(x_i)) = |y_i - f(x_i)|$

$Q(X) = \sum_{i=1}^m L(y_i, a(x_i))$ - функция риска

$Q(X) \rightarrow \min_w$

Линейная классификация

Постановка задачи линейной классификации

› **Датасет:** $\{X, Y\}$

$X = \{x_i\}_{i=1}^N$, где $x_i \in \mathbb{R}^{n_{features}}$

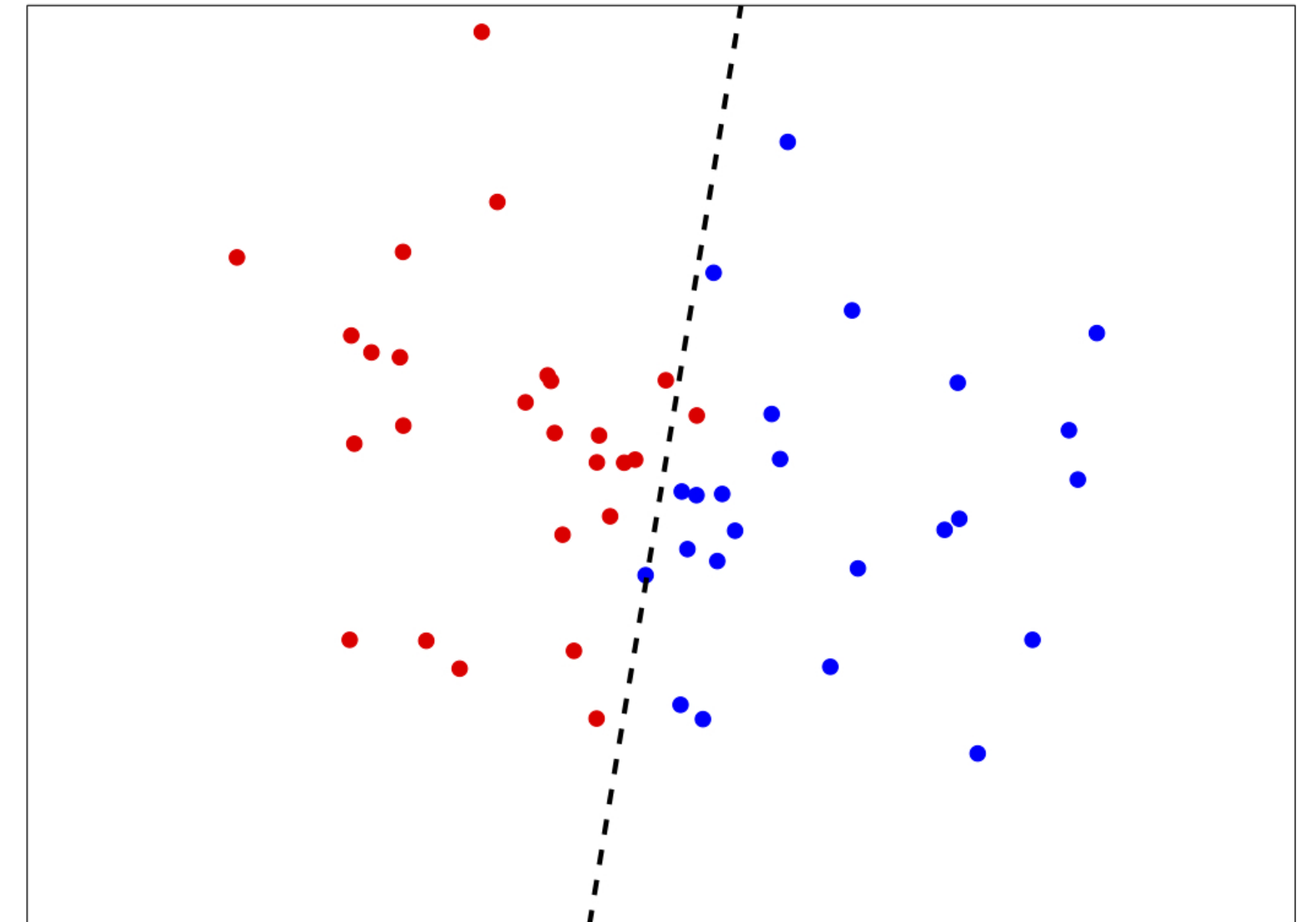
$Y = \{y_i\}_{i=1}^N$, где $y_i \in \{-1, 1\}$

› **Модель:**

› $c(x) = \text{sign}(a(x)) = \text{sign}(w_0 + x^T w) = \text{sign}(w_0 + \langle w, x \rangle)$

› **? minimization**

$$Q(f, \{X, Y\}) = \sum_{i=1}^N L(f(x_i), y_i) = \frac{1}{N} \sum_{i=1}^N ? \rightarrow \min$$



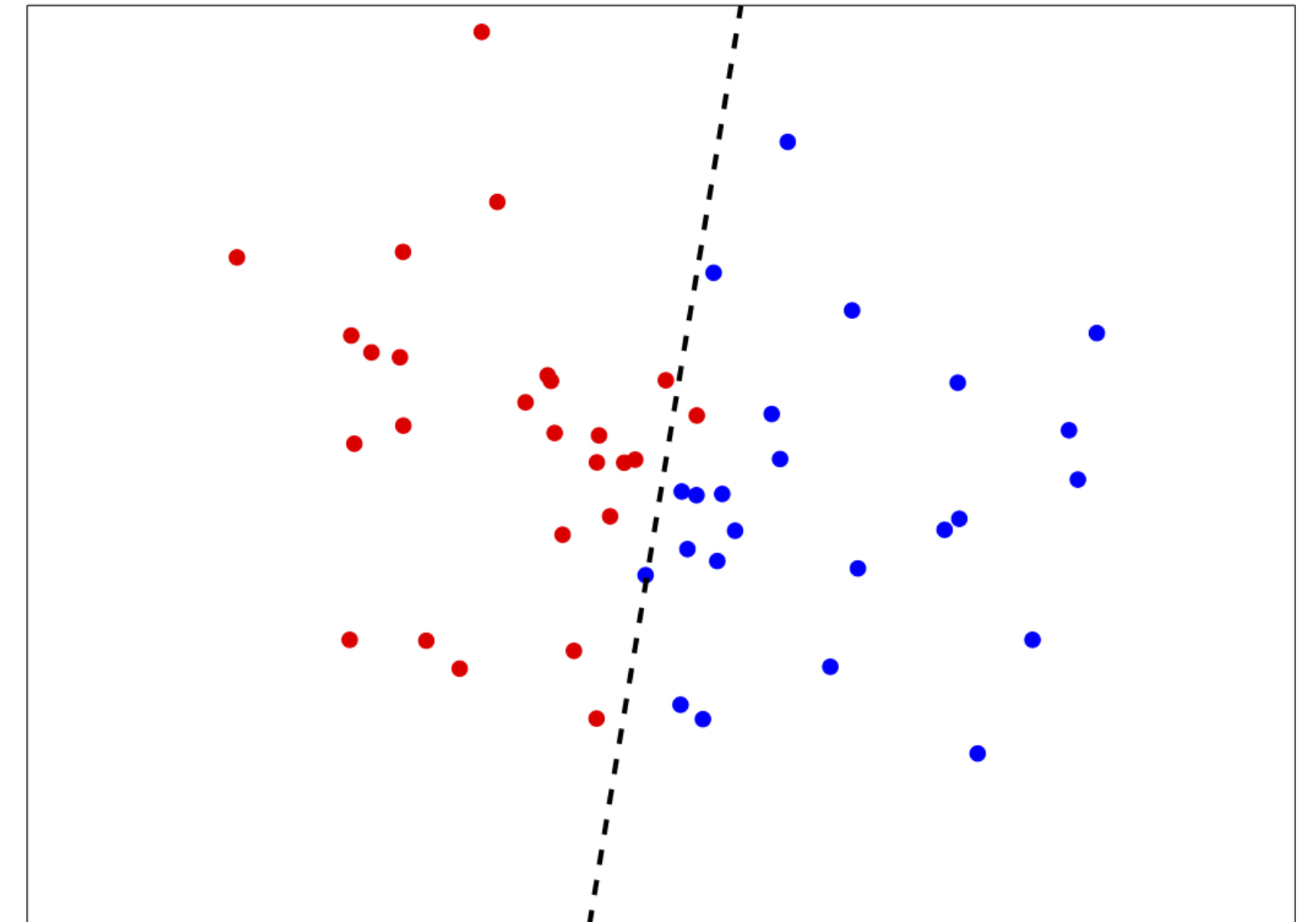
Линейная классификация, отступ

Отступ – это $M_i = M(x_i, y_i) = y_i \cdot a(x_i)$

Тогда если $M_i < 0$, то объект классифицирован неправильно.

Мы хотим минимизировать количество ошибок $\Rightarrow$

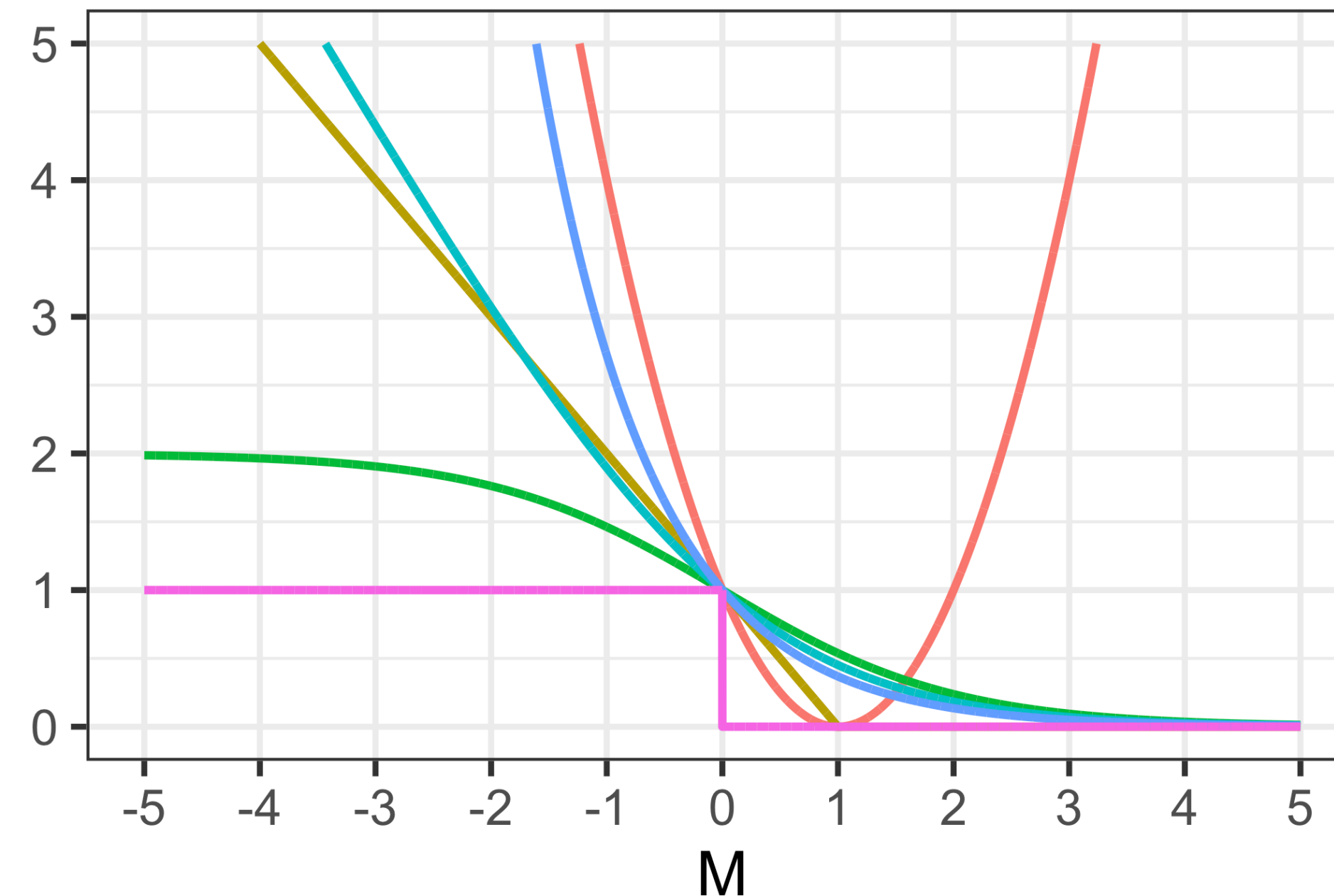
$$Q(a, \{X, Y\}) = \sum_{i=1}^N [M_i < 0] \rightarrow \min$$



Функция потерь для линейной классификации

Мы хотим минимизировать
количество ошибок $\Rightarrow$

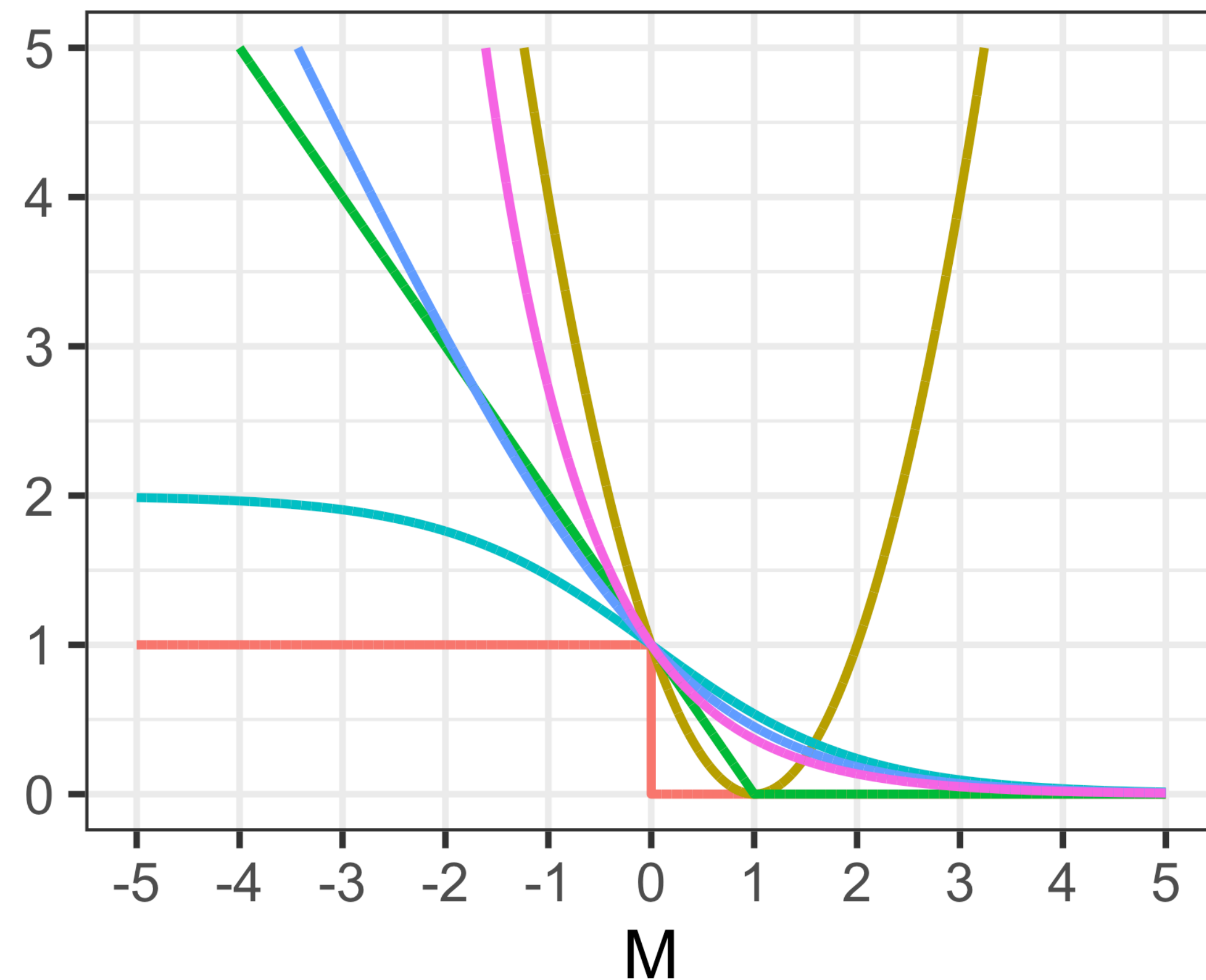
$$Q(a, \{X, Y\}) = \sum_{i=1}^N [M_i < 0] \rightarrow \min$$



Функция потерь

- Квадратичная
- Кусочно-линейная
- Сигмоидная
- Логистическая
- Экспоненциальная
- Пороговая

Функции потерь для линейной классификации



Функция потерь

- Пороговая : $[M > 0]$
- Квадратичная : $(1 - M)^2$
- Кусочно – линейная : $(1 - M)_+$
- Сигмоидная : $2(1 + e^M)^{-1}$
- Логистическая : $\log_2(1 + e^{-M})$
- Экспоненциальная : e^{-M}

2

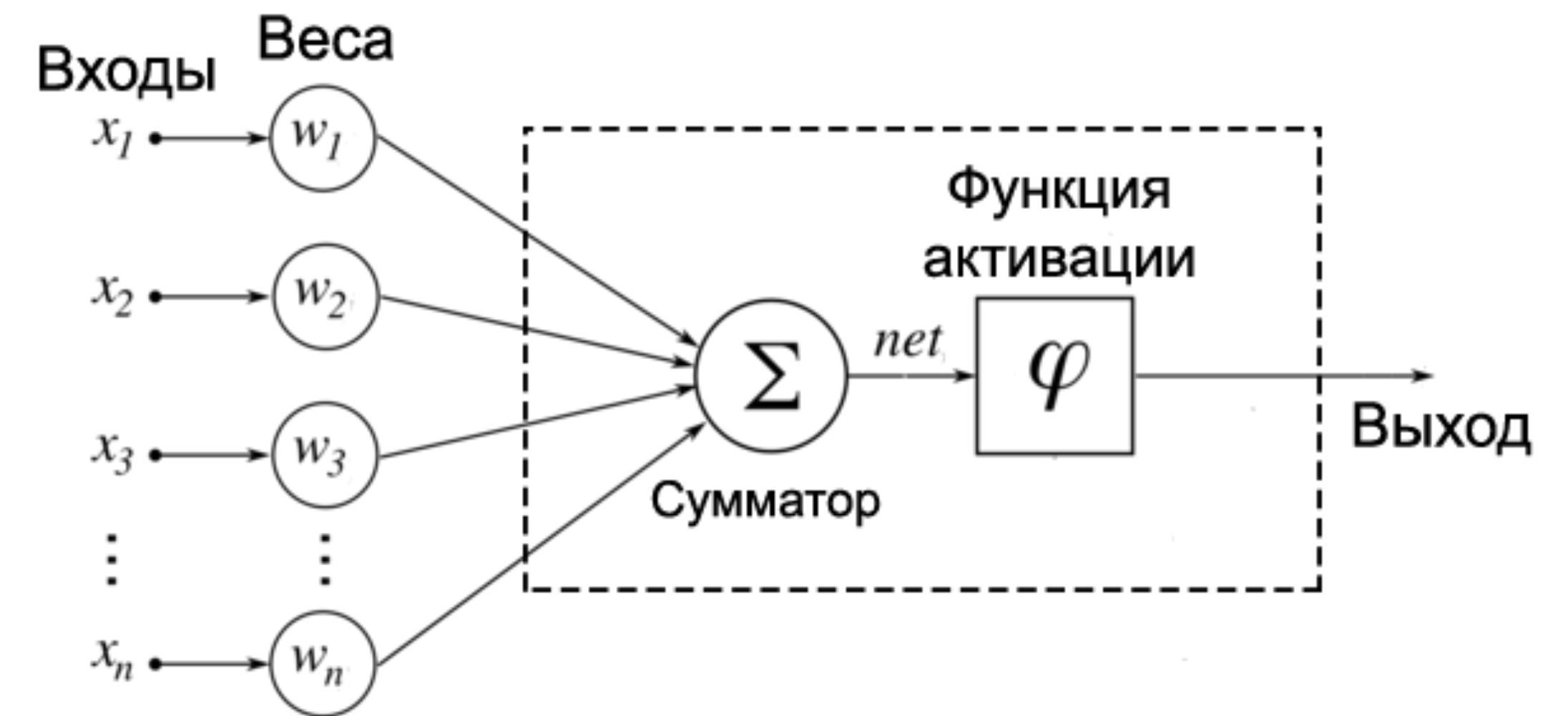


Однослойная нейронная сеть

Однослойная нейронная сеть

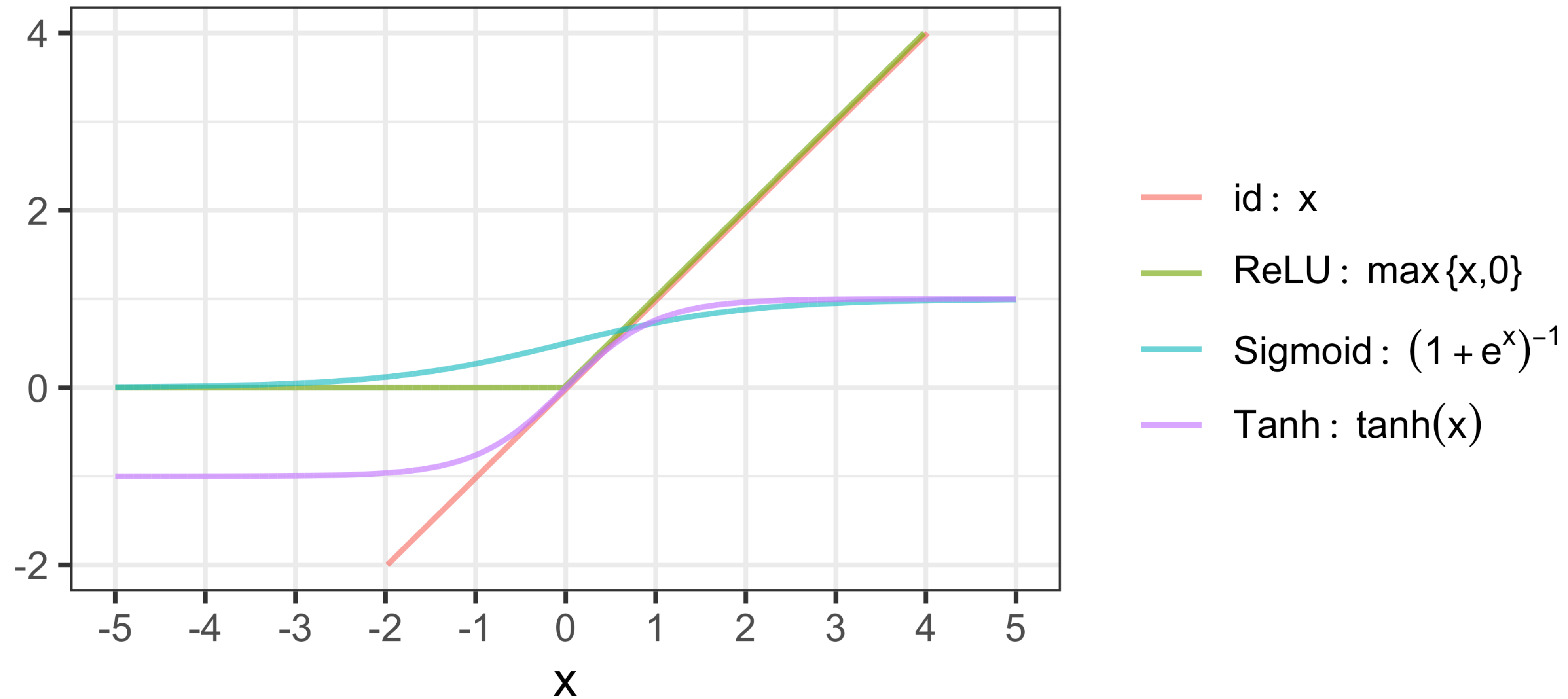
$$a(x) = \varphi(w_0 + x^T w) = \varphi(w_0 + \sum_{i=1}^{n_{features}} x_i \cdot w_i)$$

где φ – функция активации



https://neerc.ifmo.ru/wiki/index.php?title=Нейронные_сети,_перцептрон

Популярные функции активации



Логистическая регрессия

Логистическую регрессию можно представить как однослойную нейронную сеть с логистической функцией активации (сигмоидой):

$$p_i = P(y_i = 1|x_i) = \sigma(w_0 + x_i^T w) = \frac{1}{1 + e^{-(w_0 + x_i^T w)}}$$

$L_i = -(y_i \log_2(p_i) + (1 - y_i) \log_2(1 - p_i))$, где $y_i \in \{0, 1\}$ - cross-entropy loss

Сигмоида, применённая к выходу линейного слоя, позволяет получить хорошую численную оценку вероятности принадлежности объекта к классу.

Задача

Докажите утверждение с предыдущего слайда про эквивалентность постановок задачи логистической регрессии:

Постановка 1:

$$y_i \in \{-1, 1\}, M_i = M(x_i, y_i) = y_i \cdot a(x_i)$$

$$L_i = \log_2(1 + e^{-M_i})$$

Постановка 2:

$$y_i \in \{0, 1\}, p_i = P(y_i = 1|x_i) = \sigma(a(x_i)) = \frac{1}{1+e^{-a(x_i)}}$$

$$L_i = -(y_i \log_2(p_i) + (1 - y_i) \log_2(1 - p_i)) \rightarrow \min$$

Задача

Докажите утверждение с предыдущего слайда про эквивалентность постановок задачи логистической регрессии:

Постановка 1:

$$y_i \in \{-1, 1\}, M_i = M(x_i, y_i) = y_i \cdot a(x_i)$$

$$L_i = \log_2(1 + e^{-M_i})$$

Подсказка: $-\log x = \log 1/x$

Постановка 2:

$$y_i \in \{0, 1\}, p_i = P(y_i = 1|x_i) = \sigma(a(x_i)) = \frac{1}{1+e^{-a(x_i)}}$$

$$L_i = -(y_i \log_2(p_i) + (1 - y_i) \log_2(1 - p_i)) \rightarrow \min$$

3

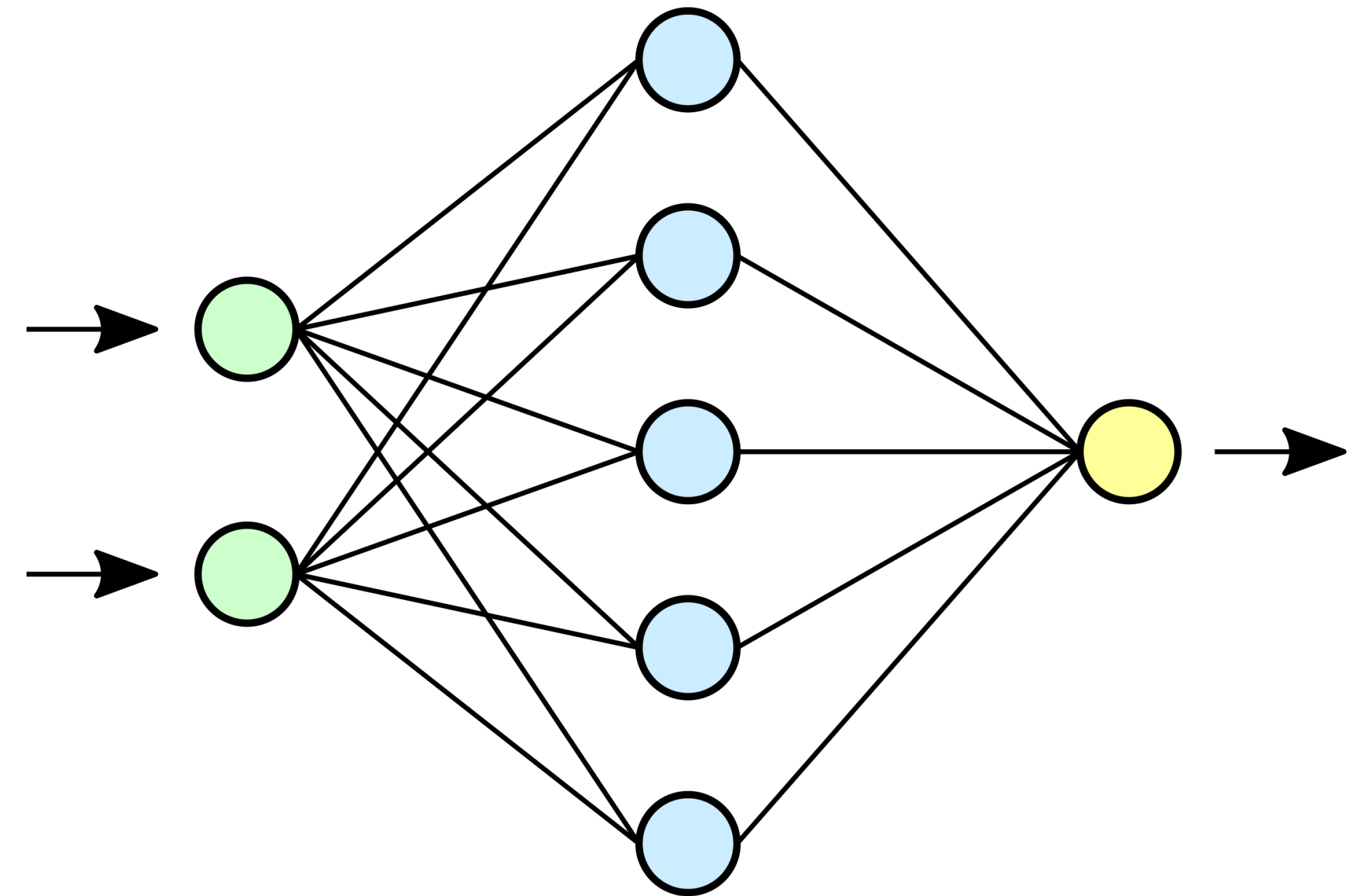


Многослойная нейронная сеть

Нейронная сеть с одним скрытым слоем

Сначала преобразуем вход из $n_{features} \rightarrow n_{hidden}$. После чего преобразуем $n_{hidden} \rightarrow n_{output}$.

Каждое из преобразований будет вида:
 $\mathbf{f}(x) = \varphi(Wx + b)$, где $W \in \mathbb{R}^{n_{in} \times n_{out}}$, $b \in \mathbb{R}^{n_{out}}$,
 φ – функция активации.



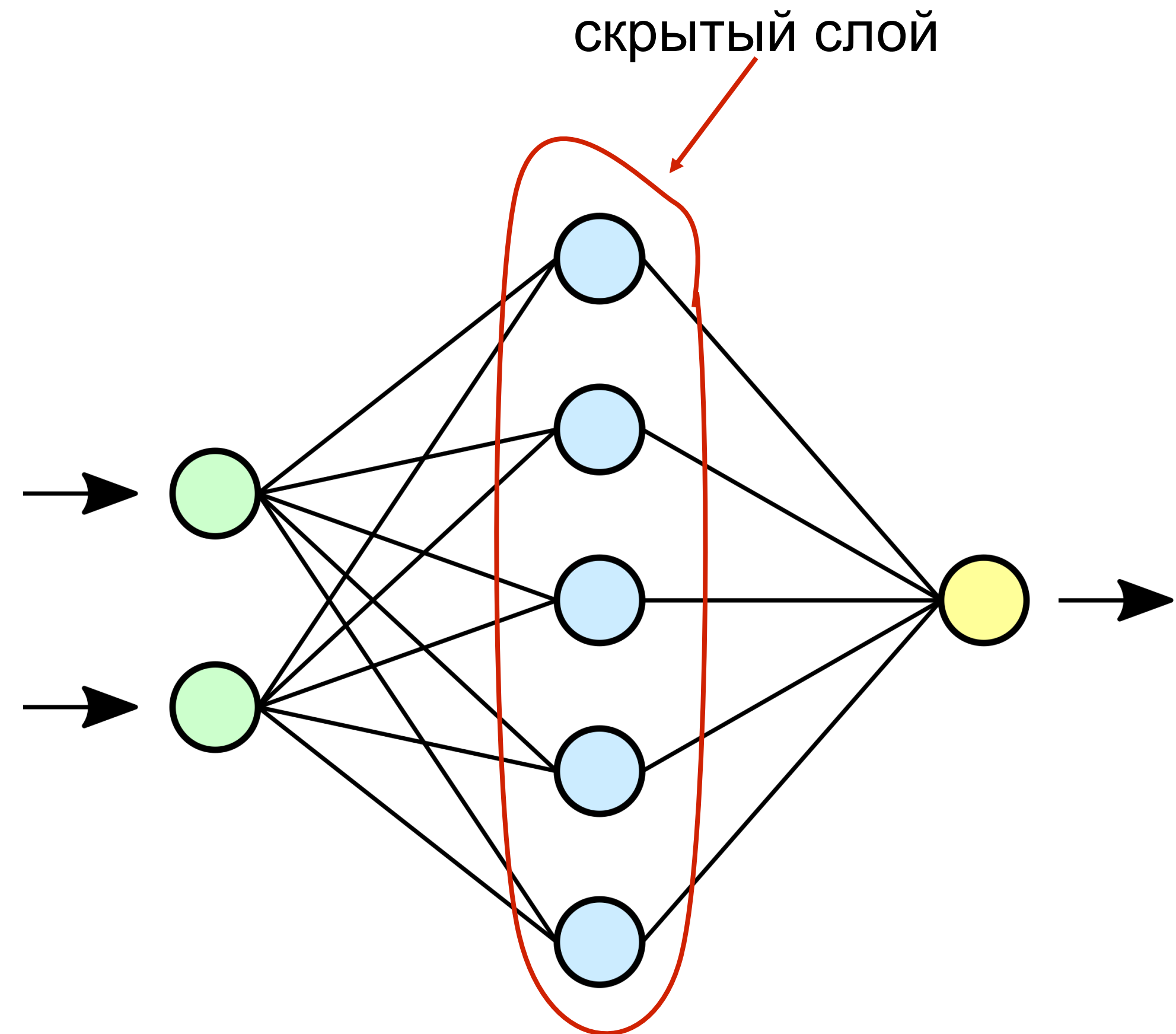
[Dake, Mysid](#), CC BY 1.0, via Wikimedia Commons

Нейронная сеть с одним скрытым слоем

Сначала преобразуем вход из $n_{features} \rightarrow n_{hidden}$. После чего преобразуем $n_{hidden} \rightarrow n_{output}$.

Каждое из преобразований будет вида:
 $\mathbf{f}(x) = \varphi(Wx + b)$, где $W \in \mathbb{R}^{n_{in} \times n_{out}}$, $b \in \mathbb{R}^{n_{out}}$,
 φ – функция активации.

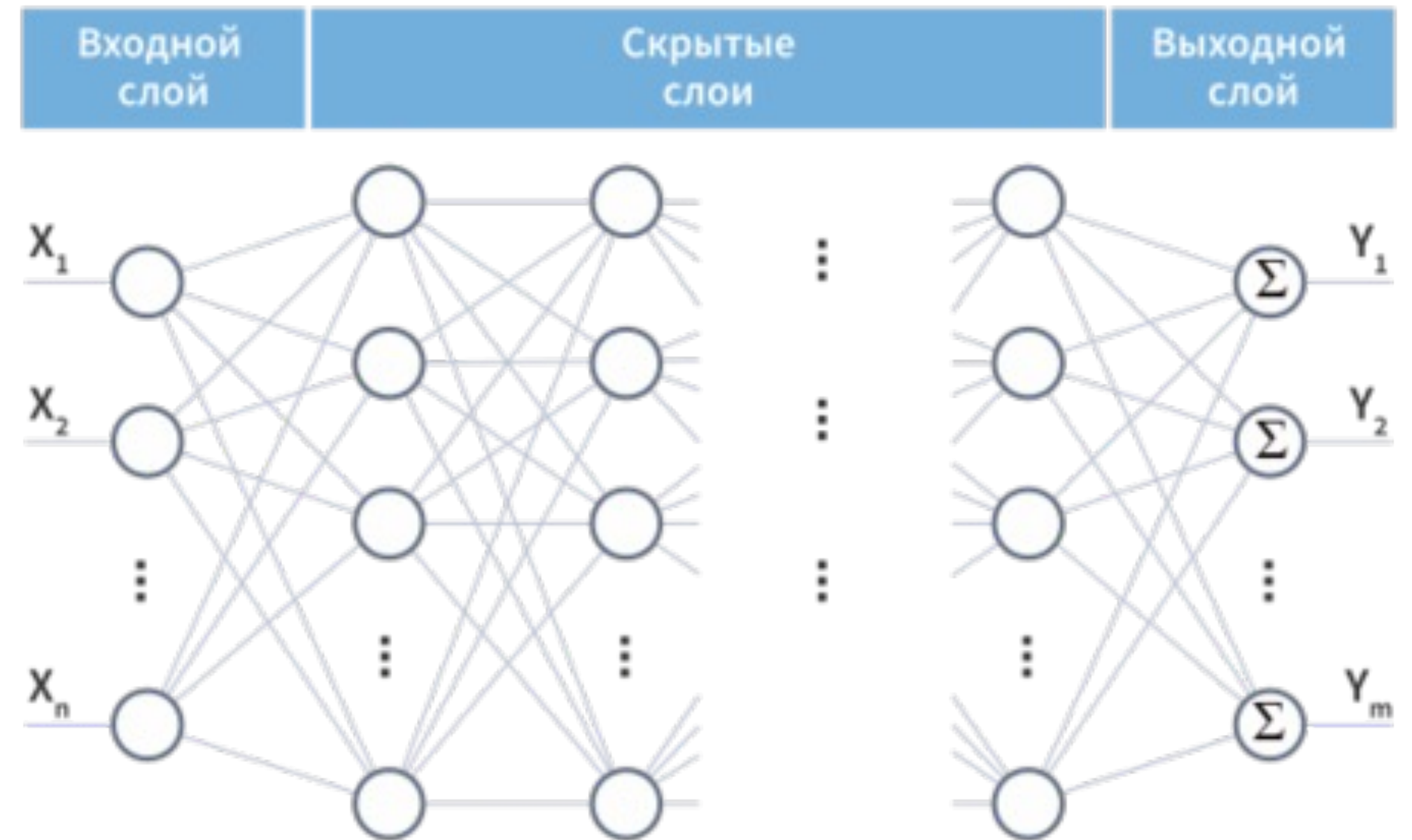
Соответственно линейная комбинация от выхода **скрытого слоя** будет выходом нейросети.



[Dake, Mysid](#), CC BY 1.0, via Wikimedia Commons

Многослойная нейронная сеть

Многослойная нейронная сеть (англ. Multilayer neural network) — нейронная сеть, состоящая из **входного**, **выходного** и расположенного(ых) между ними одного (нескольких) **скрытых слоев** нейронов.

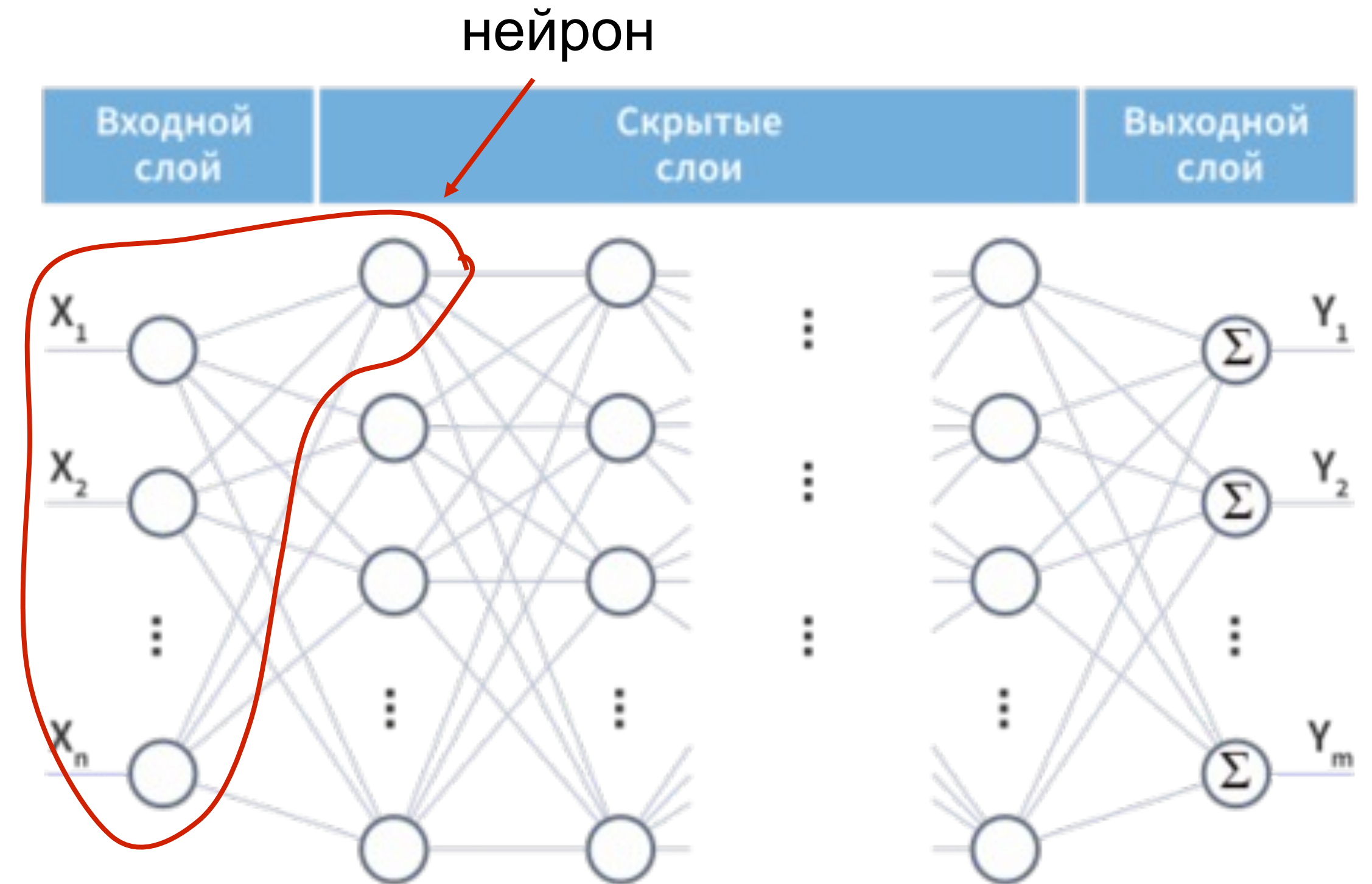


https://neerc.ifmo.ru/wiki/index.php?title=Нейронные_сети,_перцептрон

Многослойная нейронная сеть

Многослойная нейронная сеть (англ. Multilayer neural network) — нейронная сеть, состоящая из **входного**, **выходного** и расположенного(ых) между ними одного (нескольких) **скрытых слоев** нейронов.

Нейрон – преобразование $\mathbb{R}^{n_{in}} \rightarrow \mathbb{R}$ вида $\varphi(w_0 + x^T w)$.



https://neerc.ifmo.ru/wiki/index.php?title=Нейронные_сети,_перцептрон

4



Метод обратного распространения ошибки

Дифференцирование сложной функции

Цепное правило дифференцирования сложной функции для композиции функций $g(f(x))$ в одномерном случае выглядит как:

$$\frac{dg}{dx} = \frac{dg(f(x))}{dx} = \frac{dg}{df} \cdot \frac{df}{dx}$$

Дифференцирование сложной функции

Цепное правило дифференцирования сложной функции для композиции функций $g(f(x))$ в одномерном случае выглядит как:

$$\frac{dg}{dx} = \frac{dg(f(x))}{dx} = \frac{dg}{df} \cdot \frac{df}{dx}$$

Градиент — это вектор, каждая координата которого представляет собой производную функции по этой координате.

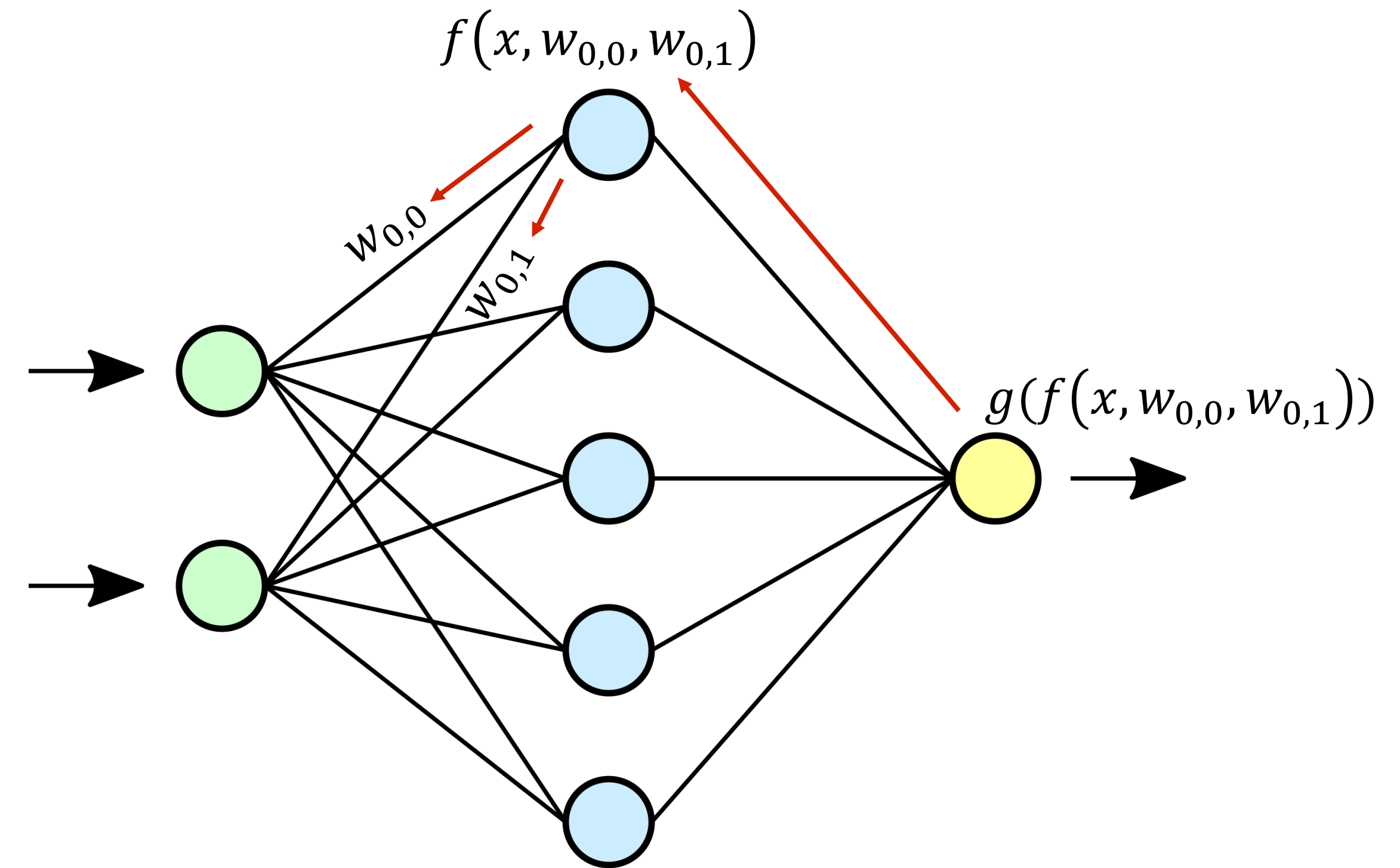
$$x \in \mathbb{R}^n, f(x): \mathbb{R}^n \rightarrow \mathbb{R}, \text{ тогда } \mathbf{\nabla} f(x) = \begin{bmatrix} \frac{\partial f}{\partial x_1}(x) \\ \vdots \\ \frac{\partial f}{\partial x_n}(x) \end{bmatrix}$$

Цепное правило работает для частных производных $\Rightarrow$ можем использовать для расчёта градиента.

Метод обратного распространения ошибки

Нам не нужно каждый раз заново считать $\frac{dL}{dw}$ для обновления каждого веса w для обновления веса в методе градиентного спуска.

Достаточно уметь вычислять градиент от следующего преобразования по предыдущему.

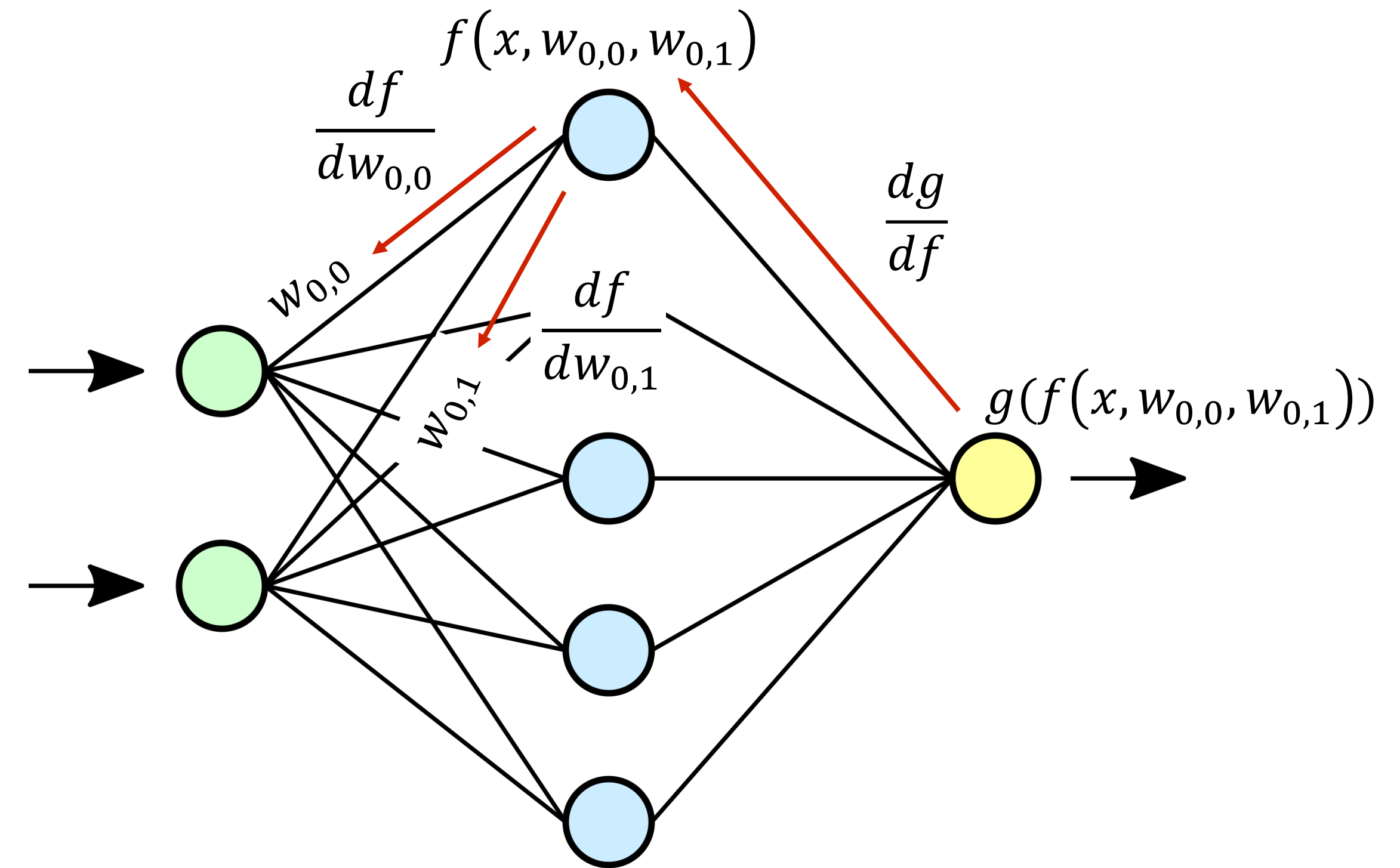


[Dake, Mysid](#), CC BY 1.0, via Wikimedia Commons

Метод обратного распространения ошибки

Нам не нужно каждый раз заново считать $\frac{dL}{dw}$ для обновления каждого веса w для обновления веса в методе градиентного спуска.

Достаточно уметь вычислять градиент от следующего преобразования по предыдущему.



[Dake, Mysid](#), CC BY 1.0, via Wikimedia Commons

5



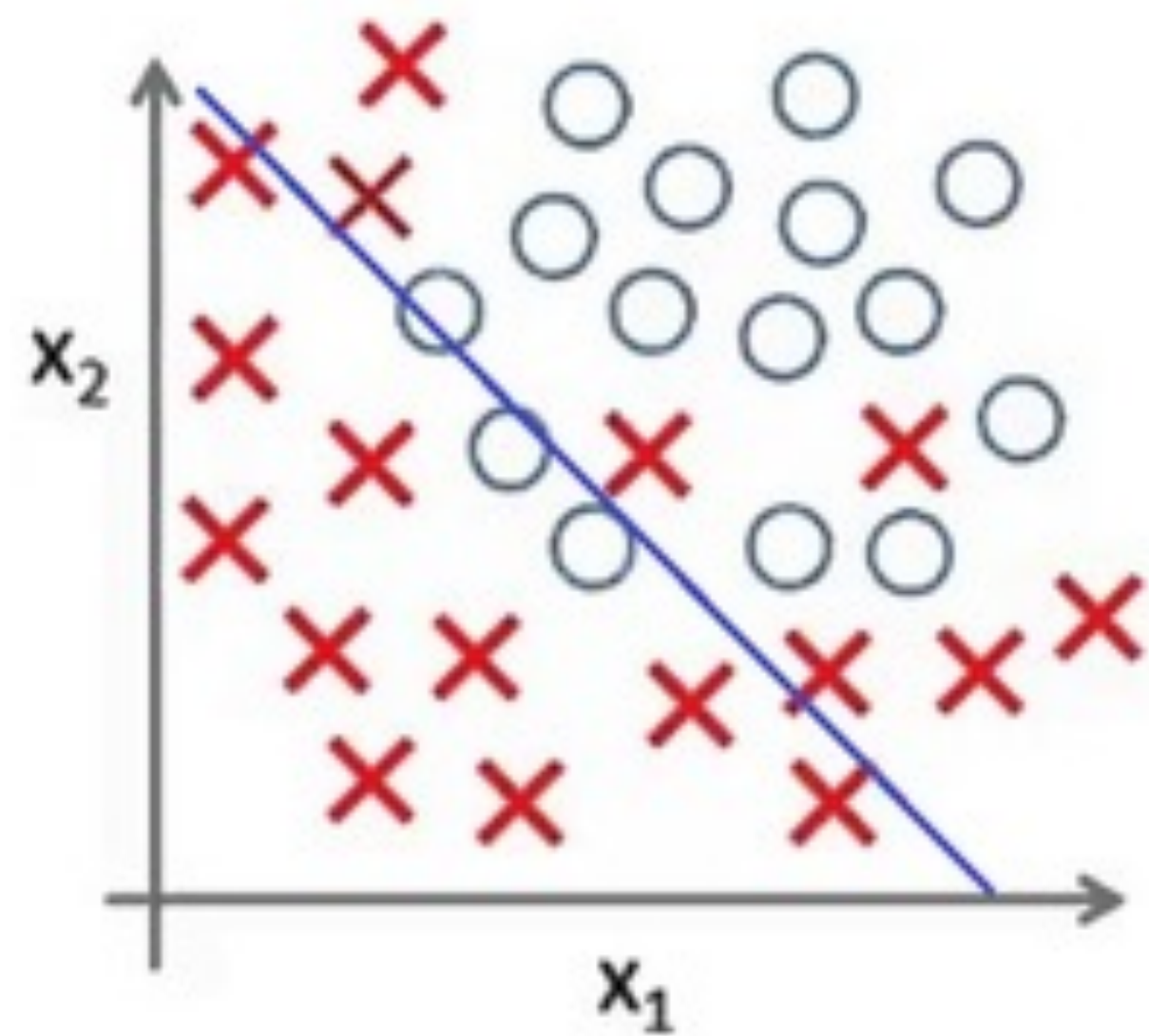
Проблема переобучения: дропаут и пакетная нормализация

Проблема переобучения (overfitting)

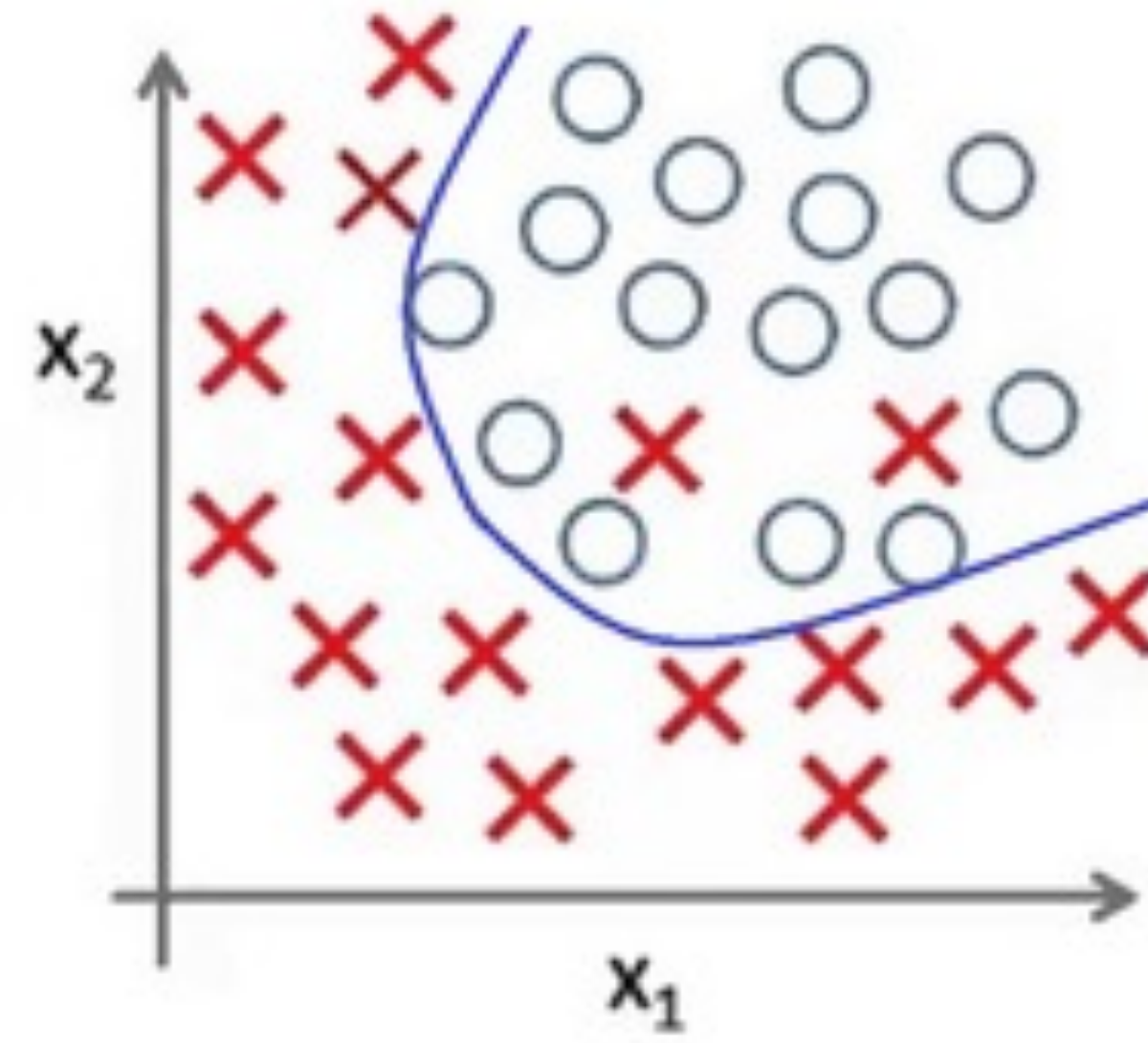
Переобучение – это одна из проблем глубоких нейронных сетей, когда алгоритм запомнил обучающую выборку, а не выучил зависимости в данных.

Переобученная модель показывает качество заметно хуже на тестовых данных, чем на обучающей выборке.

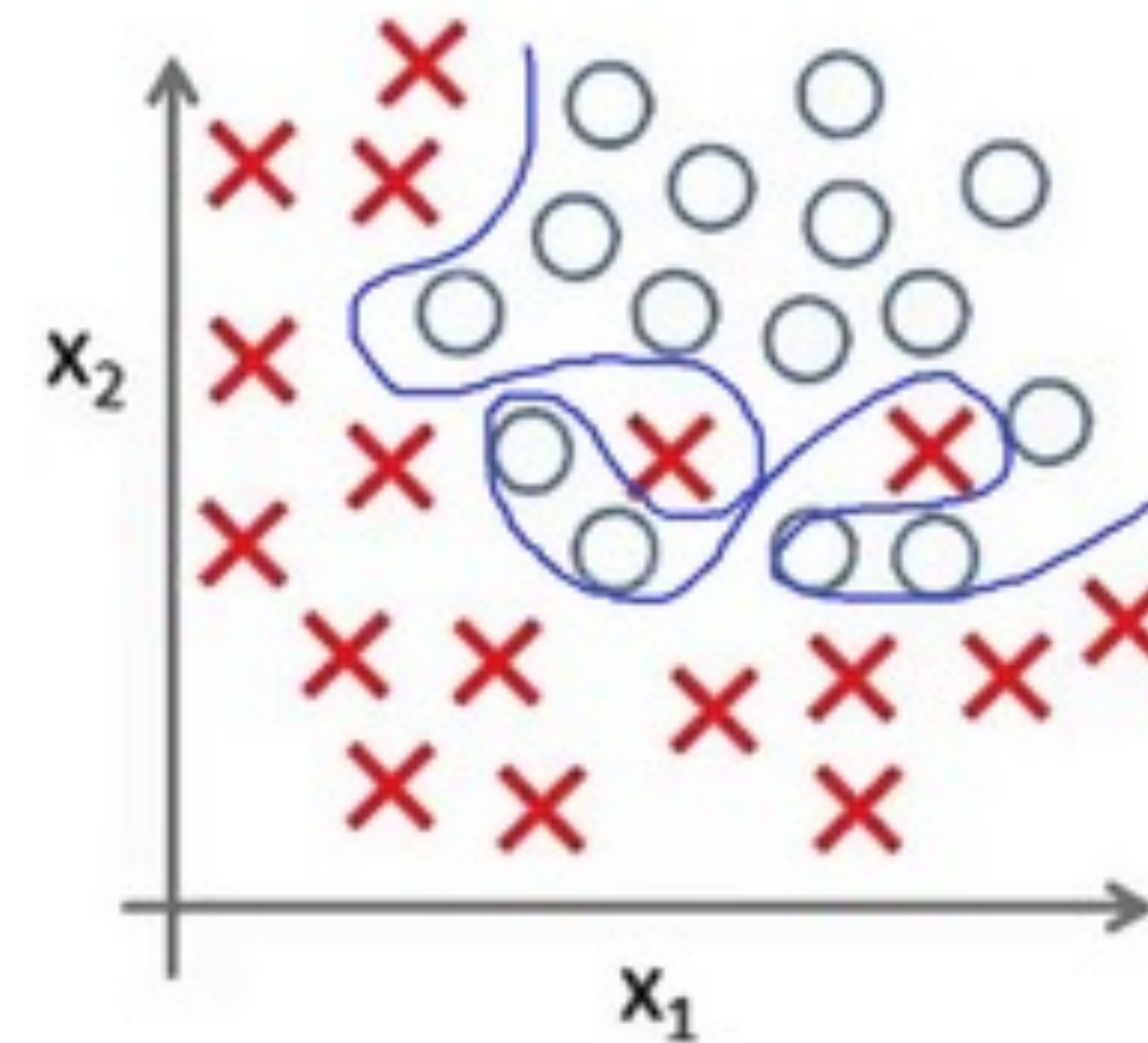
Проблема переобучения (overfitting)



Недообучение



Подходящая модель



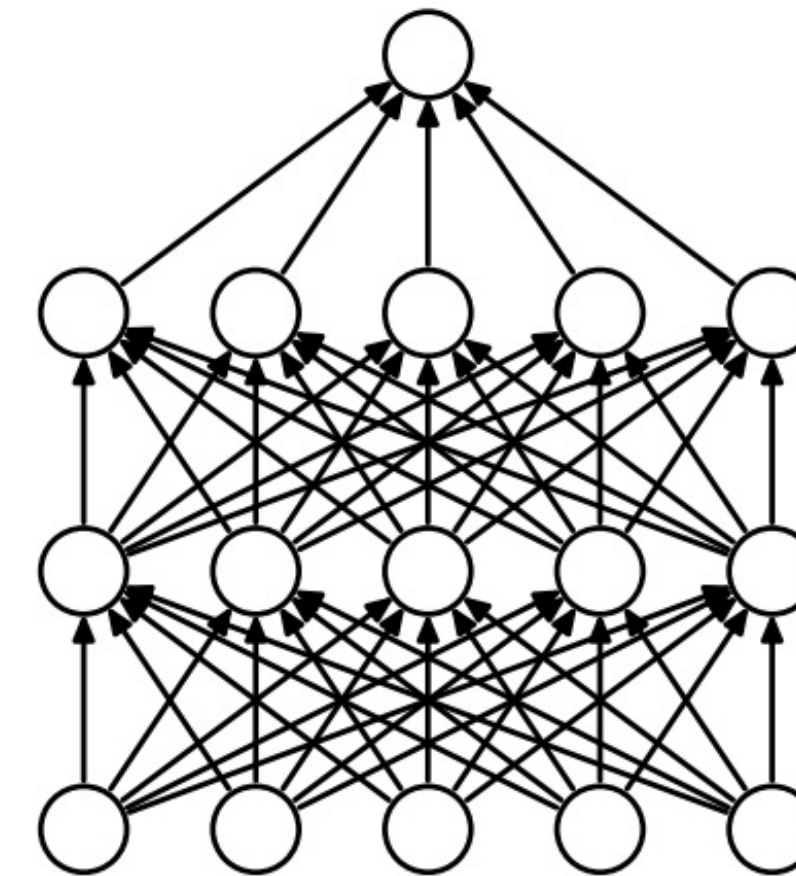
Переобучение

Метод борьбы с переобучением: дропаут

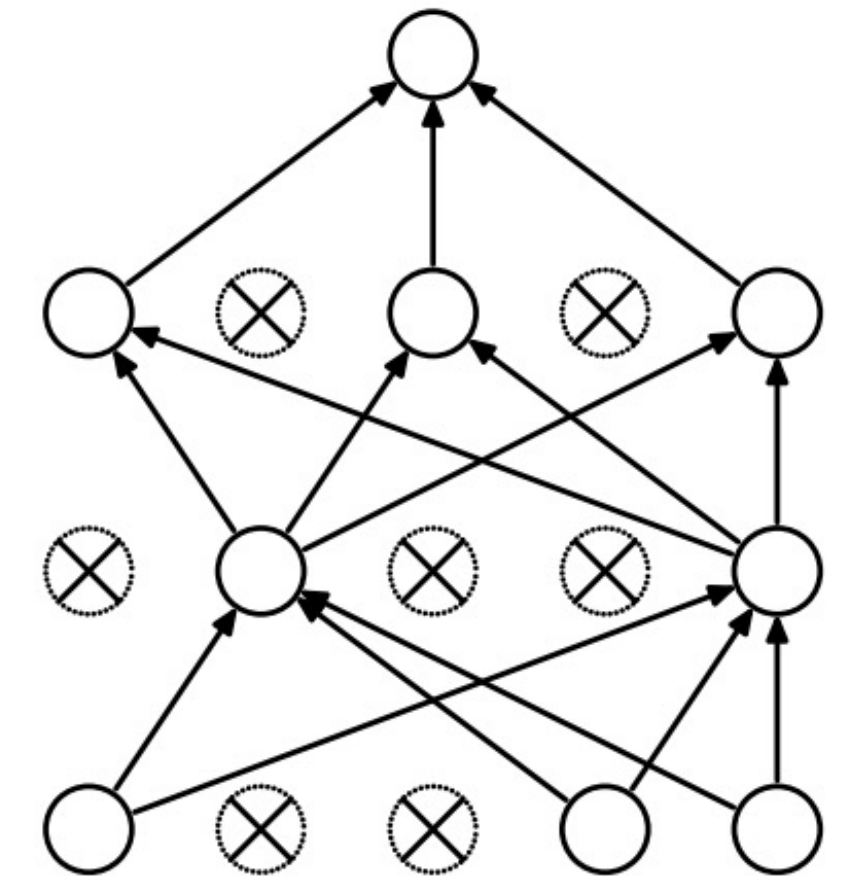
Dropout выключает на этапе обучения нейроны с вероятностью p оставляет их включенными с вероятностью $1 - p$.

Выбираем, какие нейроны выключать, независимо для каждого шага градиентного спуска.

На этапе тестирования нужно домножить выход на $(1 - p)$, чтобы компенсировать включение нейронов.



(a) Standard Neural Net



(b) After applying dropout.

<https://www.cs.toronto.edu/~rsalakhu/papers/srivastava14a.pdf>

Метод борьбы с переобучением: пакетная нормализация

Batch norm преобразует выход слоя, центрируя его и сжимая в небольшой диапазон. Таким образом гарантируется, что выходы узлов нейронной сети не будут иметь слишком больших или малых значений.

$$\hat{x}_i = \frac{x_i - \mu_{step}}{\sqrt{\sigma_{step}^2 + \epsilon}}$$

На этапе обучения мы нормируем батч (X^l из метода стохастического спуска) исходя из среднего и дисперсии батча:

$$\mu_{step} = \frac{1}{l} \sum_{i=1}^l x_i, \quad \sigma_{step}^2 = \frac{1}{l} \sum_{i=1}^l (x_i - \mu_{step})^2$$

Для использования на этапе тестирования мы запоминаем **сглаженное среднее** и **дисперсию**.



Спасибо за внимание!

6



Задачи

Задача

Докажите утверждение с предыдущего слайда про эквивалентность постановок задачи логистической регрессии:

Постановка 1:

$$y_i \in \{-1, 1\}, M_i = M(x_i, y_i) = y_i \cdot a(x_i)$$

$$L_i = \log_2(1 + e^{-M_i})$$

Подсказка: $-\log x = \log 1/x$

Постановка 2:

$$y_i \in \{0, 1\}, p_i = P(y_i = 1|x_i) = \sigma(a(x_i)) = \frac{1}{1+e^{-a(x_i)}}$$

$$L_i = -(y_i \log_2(p_i) + (1 - y_i) \log_2(1 - p_i)) \rightarrow \min$$

Задача

Модель: $\hat{y} = w_0 + w_1x_1 + w_2x_2$

Лосс: MSE

Найти: обновления весов $\frac{dL}{dw_0}$, $\frac{dL}{dw_1}$, $\frac{dL}{dw_2}$

